

STACK62

AI-Native Business Systems Platform

Product Requirements Document (PRD)

Product Name	Stack62
Category	AI-generated business software platform
Primary Vision	Enable organisations to create, extend, and share operational systems from prompts
Prepared For	Founder strategy, product design, and engineering execution

Prepared for early product planning and MVP execution

1. Executive Summary

Stack62 is an AI-native platform that allows companies to generate, extend, and manage operational software systems from natural-language requests. Rather than hardcoding a separate product for every industry, Stack62 acts as a governed business-system engine that can create domain-specific workspaces such as HR, finance, procurement, operations, CRM, inventory, compliance, school administration, real estate back office, and project management.

The platform will not rely on unsafe live self-rewriting of its production codebase. Instead, it will use an AI generation layer that converts user requests into structured system definitions such as data models, workflows, roles, dashboards, forms, automations, and agent tools. These definitions will be executed by a stable runtime and versioned so that every generated system remains secure, testable, shareable, and reversible.

Stack62 should be positioned as a cross-industry business operating platform, with AI used as the system architect and operations assistant inside each workspace.

2. Problem Statement

Most organisations still run essential workflows across disconnected spreadsheets, chat threads, ad hoc tools, and foreign SaaS products that are expensive, rigid, or poorly localised. When companies need a new internal system, they usually face three bad options: buy software that does not fit, hire developers for a long custom build, or continue operating manually.

Modern generative AI can help companies describe what they need in plain language, but most current AI products stop at text generation, copilots, or lightweight app scaffolding. Businesses still need a secure way to convert natural-language intent into usable systems with permissions, auditability, reporting, and operational logic.

Stack62 solves this by allowing organisations to request systems conversationally while the platform generates the application structure, deploys the workspace, assists users in operations, and evolves the system through governed changes.

3. Product Vision and Strategic Goals

Vision: To become the operating layer through which businesses create and run internal systems, using AI to translate intent into production-ready workflows.

- Allow a company to create a working internal system from a prompt in minutes.
- Support many industries by combining reusable core architecture with modular domain templates.
- Let authorised users evolve a system by prompting for new features, reports, workflows, or views.
- Enable system sharing as templates, cloned systems, or live shared workspaces with controlled data access.
- Make every AI-generated change reviewable, versioned, testable, and reversible.
- Use AI not only for generation but also for daily operation within each workspace.

4. Product Principles

- Governed generation over uncontrolled self-editing.
- Structured definitions over arbitrary code whenever possible.
- Cross-industry adaptability with reusable primitives.
- Security, roles, and auditability as first-class features.

- Human approval for risky changes and production-impacting updates.
- Shareability and repeatability so generated systems can become reusable assets.

5. Core Product Concept

A tenant on Stack62 can create one or more named systems. Each system is a business workspace generated and managed through AI. For example, an organisation may create PeopleFlow HR, Atlas Finance, or FieldOps Procurement as separate systems within one company account.

Each generated system contains the following layers:

- System manifest: name, purpose, modules, visibility, and lifecycle status.
- Data schema: entities, fields, relationships, constraints, and computed fields.
- Roles and permissions: workspace roles, page access, action rights, and field masking.
- Views: dashboards, forms, tables, cards, calendars, kanban boards, charts, and reports.
- Workflow rules: approvals, assignments, escalations, triggers, reminders, and state transitions.
- Agent tools: AI actions that can read, create, summarise, recommend, or automate work inside the system.
- Sharing model: template-only, cloned copy, or shared live workspace with role-based access.

6. Users and Personas

Persona	Description	Primary Need	Success Signal
Founder / Business Owner	Needs systems quickly without managing a full software team.	Fast setup, low cost, flexibility	Operational system live within one day
Ops / Admin Manager	Runs the workflows and wants approvals, reports, and accountability.	Reliable daily operations	Reduced manual coordination and fewer process leaks
Department Lead	Needs a specialised workspace for HR, finance, procurement, or projects.	Domain-specific usability	Team adoption and decision-ready reporting
IT / Product Admin	Governs configuration, permissions, integrations, and change reviews.	Control and security	Stable, auditable evolution of systems
External Collaborator	May need limited access to a specific workflow or dashboard.	Safe partial access	Can participate without seeing unnecessary data

7. Jobs To Be Done

- When I describe a system I need, I want Stack62 to generate a usable workspace so I do not need to design it from scratch.
- When my business process changes, I want to request modifications conversationally so the system can evolve without a full rebuild.
- When my team grows, I want to grant role-based access to specific people and modules so sensitive data stays protected.
- When I discover a useful configuration, I want to save and reuse it as a shareable system template or cloned solution.
- When AI makes changes, I want to understand what changed and be able to approve, reject, or roll back those changes.

8. Functional Requirements

8.1 Workspace Generation

- User can create a new system from a prompt, template, or copied system.
- System creation flow captures name, purpose, industry, team size, and governance level.
- AI proposes modules, entities, fields, views, workflows, and permissions.
- User can accept, edit, or regenerate before publishing the first version.

8.2 System Extension

- Authorised users can request additional modules, dashboards, reports, forms, or automations.
- AI change proposals must describe affected schema, views, workflows, and permissions.
- High-impact changes should require review before going live.
- System versions must support rollback to a stable previous state.

8.3 Runtime Experience

- Each system should support dashboard pages, record pages, forms, analytics, tasks, and search.
- The runtime must render interfaces dynamically from structured definitions.
- Users should be able to filter, sort, group, and export permitted data.
- The platform should support web-first responsive usage, with mobile-friendly views for common workflows.

8.4 AI Operations Assistant

- Each system includes an embedded AI assistant aware of the system schema, roles, workflows, and approved tools.
- The agent can answer questions, create records, summarise work, suggest actions, and trigger allowed automations.
- The assistant must respect role permissions and never access restricted data.
- All agent actions must be logged with actor identity and timestamp.

8.5 Sharing and Distribution

- A system can be shared as template-only, cloned instance, or live shared workspace.
- Owners can decide whether data is included, masked, or excluded entirely.
- Invite flows should support role assignment, expiry rules, and revocation.
- Reusable systems should be discoverable inside a private organisation library and eventually a public marketplace.

8.6 Governance and Auditability

- Every AI-generated or user-approved change must be versioned.
- Admins must be able to compare versions and inspect change summaries.
- Audit logs must cover data changes, permission changes, workflow changes, and AI actions.
- Sensitive environments should support draft mode, staging preview, and approved publish.

9. Non-Functional Requirements

Area	Requirement	Target Direction
------	-------------	------------------

Area	Requirement	Target Direction
Performance	Normal views should load quickly for standard SME-sized datasets.	Sub-3 second common page loads under expected load
Scalability	Support many tenants and many systems per tenant.	Multi-tenant architecture with workspace isolation
Availability	Business workspaces must be dependable for daily use.	Production monitoring, backups, recovery
Security	Role-aware access and secure storage are mandatory.	Encryption, least privilege, audit logs
Reliability	Generated systems must not break live operations.	Validation engine, staging, rollback
Explainability	Users should understand what AI changed.	Change diff, rationale summary, approval flow

10. Product Architecture

Stack62 should be built as a layered platform, not as a chatbot modifying arbitrary production code on demand.

- Core Platform Layer: tenant management, authentication, billing, storage, audit, integrations, notifications, version history.
- Generation Layer: prompt interpreter, domain planner, schema generator, view generator, workflow generator, permission generator.
- Validation Layer: policy checks, schema sanity checks, migration analysis, conflict detection, risk scoring, test generation.
- Runtime Layer: UI renderer, query engine, workflow executor, report engine, search, file handling, agent tool execution.
- Governance Layer: approval flows, changelogs, rollback, environment promotion, observability, security controls.

11. Internal Object Model

To keep the platform controllable, each generated system should compile from structured internal objects instead of unconstrained raw code.

- Tenant
- Workspace / System
- Module
- Entity
- Field
- Relationship
- View Definition
- Workflow Definition
- Automation Rule
- Permission Policy
- Agent Tool
- Version Snapshot
- Audit Event

- Share Package / Template

12. Example User Flow

Illustrative scenario: A company prompts Stack62 to create an HR system and later extends it into finance operations.

	Action
Step 1	Admin enters request: 'Create an HR management system for a 60-person company with attendance leave, onboarding, and reporting.'
Step 2	AI proposes modules, entities, dashboards, workflows, and default roles.
Step 3	Admin reviews the generated system plan and publishes version 1.
Step 4	Team begins using the HR workspace with role-based access.
Step 5	Admin later prompts: 'Add expense approvals, vendor payments, and monthly finance reporting.'
Step 6	AI proposes new finance modules, linked permissions, and dashboards; changes go through review.
Step 7	Admin approves the extension, names the full system 'Northstar Operations Suite', and shares it with finance and operations leads.
Step 8	A template copy can now be reused in another business unit without copying the live data.

13. Security, Permissions, and Data Access

- Tenant isolation must separate company environments completely.
- Role-based access control should be supported at workspace, module, page, record, and field level.
- Masked field views should allow selective visibility for sensitive attributes such as salary, payments, or personal identifiers.
- Agent permissions must be derived from human permissions, with additional tool restrictions where needed.
- Share settings must clearly distinguish between shared live data, cloned data, empty cloned structure, and template-only sharing.
- Backups, retention policies, and access logs must be configurable for serious business use.

14. AI Behavior and Guardrails

- AI should generate within predefined platform primitives first, using code only when necessary and allowed.
- High-risk schema changes should trigger warnings and approval requests.
- The model should explain the intended effect of any proposed change in plain language.
- Generated workflows must pass validation checks before publication.
- AI should never silently expose restricted data or weaken permissions.
- The assistant must show clear confidence boundaries when it is unsure.

15. MVP Scope

The MVP should prove the core thesis: a company can describe a system, receive a usable workspace, operate on it, and safely evolve it over time.

15.1 In Scope for MVP

- Single-tenant organisation account with multiple named systems.
- Prompt-to-system generation for selected domains: HR, CRM, project operations, inventory, procurement tracker, and finance tracker.
- Generated entities, forms, tables, dashboards, and simple charts.
- Basic workflow rules: approvals, reminders, assignments, status transitions.
- Role-based access at workspace and module level, with limited field-level controls.
- Embedded AI assistant for querying records, creating items, summarising, and guiding admins.
- Version history, draft changes, manual approval, and rollback.
- System sharing as template-only and cloned instance.

15.2 Out of Scope for MVP

- Arbitrary unrestricted code generation into live production.
- Complex public marketplace of third-party templates.
- Highly advanced cross-system orchestration.
- Deep ERP-level accounting logic across every domain.
- Industry-specific compliance certification out of the gate.

16. Roadmap

Phase	Objective	Key Deliverables
Phase 1	Prove prompt-to-system generation	Schema generation, dynamic UI renderer, basic templates, manual publish
Phase 2	Operationalise daily usage	Approvals, dashboards, notifications, AI assistant, exports, audit logs
Phase 3	Support safe evolution	Diff views, staging, rollback, risk scoring, migration engine
Phase 4	Expand sharing and reuse	Template library, cloning, internal marketplace, access packages
Phase 5	Deepen power and moat	Integrations, plugins, advanced automations, cross-system agent orchestration

17. Success Metrics

- Time from prompt to first published system.
- Percentage of generated systems published after first draft.
- Frequency of post-launch system extensions by prompt.
- User activation and weekly active usage inside generated systems.
- Share and clone rate of systems and templates.
- Rollback rate and failed change rate.
- Net retention of paying organisations.

18. Key Risks and Mitigations

Risk	Impact	Mitigation
Uncontrolled schema sprawl	Messy systems and hard maintenance	Use templates, validation, schema diffs, and reviewable migrations
Permission leakage	Sensitive data exposure	Field-level policies, share previews, audit logs, strict defaults
Poor generated usability	Low adoption	Domain templates, UI heuristics, preview/edit loop before publish
Model hallucination in system changes	Broken logic or trust loss	Guardrails, validation engine, staging, human approval
Too-broad positioning	Weak go-to-market focus	Launch with narrow initial verticals and reusable templates

19. Recommended Initial Positioning

Although Stack62 is designed as a cross-industry platform, the launch message should not be 'build anything for anyone'. The better strategy is to enter through one or two sharp use cases while keeping the architecture cross-industry underneath.

- SME back-office systems for HR, operations, and approvals.
- Industry starter packs for real estate, schools, logistics, and service businesses.
- Positioning line: 'Create the business system you need by describing it.'

20. Recommended Technical Stack

- Frontend: Next.js / React with a dynamic component renderer driven by JSON or schema definitions.
- Backend: FastAPI or Node.js service layer for generation, runtime orchestration, and APIs.
- Database: PostgreSQL for transactional data, metadata, and versioned system definitions.
- Cache / queues: Redis and background workers for async jobs and automations.
- Storage: object storage for uploads, exports, and template bundles.
- AI orchestration: model gateway plus task-specific planners, generators, validators, and tool-execution agents.
- Observability: structured logs, tracing, audit event stream, and admin monitoring dashboard.

21. Open Product Questions

- How much editable freedom should end users have before admin approval is required?
- Should field-level access control be included in MVP or phase two?
- How much domain-specific logic should be encoded as templates versus generated fresh each time?
- What should be the first beachhead vertical for launch and sales validation?
- What share modes should be monetised as premium features?
- How will Stack62 handle custom code safely in later phases?

22. Conclusion

Stack62 is best understood as an AI-generated business systems platform rather than a self-editing app. Its value comes from turning user intent into governed operational software that can be created quickly, extended safely, and reused across teams and industries. If built correctly, Stack62 can become the layer through which organisations generate and run internal software without waiting for traditional development cycles.

The immediate next step after this PRD should be a product blueprint for MVP execution covering information architecture, object model, UI map, prompt-to-system flow, data structures, and engineering milestones.